

Capítulo 5: Estructuras de Datos y Colecciones Funcionales

1. Explicación Teórica: Inmutabilidad y HOFs

El modelado de datos en Scala, especialmente con colecciones, se rige por la filosofía de la **Programación Funcional (PF)**, priorizando la inmutabilidad y la composición.

Inmutabilidad como Pilar Central

A diferencia de Java o Python, donde las estructuras de datos suelen ser mutables por defecto, Scala promueve colecciones que son intrínsecamente **inmutables**. Esto significa que, si se aplica una operación de "modificación" a una lista, el resultado no es un cambio en la lista original, sino la **creación de una nueva colección** que incorpora el cambio. Esta práctica es crucial para construir **sistemas concurrentes seguros**, ya que elimina el riesgo de condiciones de carrera (*race conditions*) y simplifica drásticamente la lógica del programa.

- **Comparativa (Java/Python):** En lenguajes imperativos, se usaría un bucle para modificar una lista o array en su lugar (*in-place*), lo cual es mutabilidad. En Scala, esto se modela como una **transformación declarativa** que produce un nuevo valor inmutable.

Funciones de Orden Superior (HOFs)

Las **Funciones de Orden Superior (HOFs)** son el mecanismo principal para interactuar y transformar colecciones. En lugar de escribir lógica imperativa (*cómo* iterar), se define lógica declarativa (*qué* transformación aplicar a los datos). Las HOFs clave, como `map`, `filter`, `fold` y `reduce`, tratan las funciones como valores de primera clase para componer manipulaciones de colecciones.

Consistencia de la API de Colecciones

Una ventaja significativa del diseño de Scala es que las operaciones funcionales (como `map` o `filter`) son **consistentes** a lo largo de todas las colecciones principales (`List`, `Set`, `Map`, `Vector`, `Array` o incluso `Option`). Una vez que se aprende a usar `map` en una `List`, el mismo método funciona de manera coherente en un `Set` o un `Map`.

2. Colecciones Inmutables Fundamentales y Sintaxis

Scala distingue claramente entre colecciones mutables (como `Array` y `scala.collection.mutable.Set`) y las **inmutables** (por defecto, las más utilizadas).

A. Listas (List[T])

La estructura fundamental y preferida para colecciones ordenadas. Son inmutables y se optimizan para la recursión y las operaciones de anteposición (agregar al inicio).

Característica	Sintaxis Idiomática de Scala
Creación (Factory)	<code>val nums = List(1, 2, 3)</code> (Usa <code>apply</code> en el <i>Companion Object</i>).
Lista Vacía	<code>val vacia = List.empty[Int]</code> o <code>val vacia = Nil</code> .
Anteposición (Cons)	<code>val nueva = 0 :: nums</code> (Rápido, devuelve <code>List(0, 1, 2, 3)</code>). El operador <code>::</code> es asociativo a la derecha.
Acceso a Elementos	<code>nums.head</code> (Primer elemento), <code>nums.tail</code> (Resto de la lista).

Ejemplo de List (Modelado de Ruta)

```
1 // Definición de una lista de puntos de un recorrido GPS inmutable
2 val rutaInicial = List((0, 0), (5, 10), (12, 15))
3
4 // Intento de modificar la lista (Error de compilación)
5 // rutaInicial(0) = (1, 1)
6
7 // Transformación (Crea una nueva lista inmutable)
8 val nuevaRuta = (0, 0) :: rutaInicial.tail // Agrega (0, 0) al inicio, sin
    mutar rutaInicial
9 // nuevaRuta: List[(Int, Int)] = List((0, 0), (5, 10), (12, 15))
```

B. Conjuntos (Set[T])

Colecciones inmutables que no permiten valores duplicados y no garantizan un orden específico. Se usan para membresía (verificar si un elemento existe).

```
1 // Un Set solo contendrá valores únicos: 1, 2, 3
2 val idsUnicos = Set(1, 2, 3, 2, 1)
3
4 // Operación matemática: Intersección (Intersect)
5 val otrosIds = Set(3, 4, 5)
6 val enComun = idsUnicos.intersect(otrosIds) // Set(3)
```

C. Mapas (Map[K, V])

Asociaciones de clave-valor donde las claves son únicas. Internamente, un mapa es una colección de tuplas (pares de clave y valor).

Característica	Sintaxis Idiomática de Scala
Creación	<code>Map("A" -> 1, "B" -> 2)</code> (Usa el operador <code>-></code> para crear tuplas (clave, valor)).
Acceso Seguro	<code>mapa.get("A")</code> (Devuelve un <code>Option[V]</code> para evitar <code>NullPointerException</code>).
Acceso Directo (Inseguro)	<code>mapa("A")</code> (Llama al método <code>apply</code> ; lanza excepción si la clave no existe).

Ejemplo de `Map` (Configuración de Servicios)

```
1 // Mapa inmutable de configuraciones (String -> Int)
2 val puertosServicio = Map(
3     "AuthService" -> 8080,
4     "DataService" -> 9000
5 )
6
7 // Acceso seguro: devuelve Option[Int] (Some(8080) o None)
8 val puertoAuth = puertosServicio.get("AuthService")
9
10 // Uso funcional del Option para obtener el valor o un default
11 val puertoDesconocido =
12     puertosServicio.get("ConfigService").getOrElse(5000)
13 // puertoDesconocido: 5000
```

3. Operaciones Funcionales Clave (HOFs)

Las HOFs permiten la transformación y composición de datos sin recurrir a variables mutables o bucles imperativos,.

A. Transformación (`map`)

Aplica una función a cada elemento de la colección y devuelve una **nueva colección** con los resultados,. El tamaño de la colección se mantiene, pero el tipo de los elementos puede cambiar.

```
1 // Lista de códigos de estado HTTP
2 val codigos = List(200, 404, 503, 301)
```

```

3
4 // Aplicar un mapeo para convertir códigos a Strings descriptivos
5 val descripciones = codigos.map(codigo => s"HTTP Status: $codigo")
6 // List("HTTP Status: 200", "HTTP Status: 404", ...)

```

B. Filtrado (`filter` y `flatMap`)

`filter` retiene solo los elementos para los cuales una función (llamada **predicado**) devuelve `true`.

```

1 // Predicado: una función que devuelve un Boolean
2 val esError = (codigo: Int) => codigo >= 400
3
4 // Filter: solo mantiene los códigos de error (404, 503)
5 val errores = codigos.filter(esError)
6
7 // Uso de shorthand para filtrar implícitamente
8 val erroresServidor = codigos.filter(_ >= 500) // List(503)

```

`flatMap` : Es la combinación de `map` (transformación) seguida de `flatten` (aplanamiento), y es fundamental para transformar cada elemento en **cero o más elementos** dentro de una única colección plana. Si un elemento se mapea a una colección vacía (por ejemplo, `List.empty`), ese elemento se elimina del resultado final.

```

1 // Si el código es un error (>=400), lo duplica; si no, lo ignora (Map vacío)
2 val duplicarErrores = codigos.flatMap {
3     case c if c >= 400 => List(c, c)
4     case _ => List.empty[Int]
5 }
6 // duplicarErrores: List(404, 404, 503, 503)

```

C. Reducción (`fold` y `reduce`)

Estas funciones resumen toda una colección en un **único valor**.

- `reduce` : Requiere que la colección no esté vacía y utiliza el primer elemento como valor inicial (semilla).
- `fold` (o `foldLeft` / `foldRight`) : Permite especificar un **valor inicial o semilla** (`seed`) explícito, lo que lo hace más seguro y común en la PF.

```

1 // Se inicia la suma en 100.
2 val sumaTotal = codigos.foldLeft(100) { (acumulador, elemento) =>
3     acumulador + elemento

```

```
4 }
5 // sumaTotal: 100 + 200 + 404 + 503 + 301 = 1508
```

4. Iteración Declarativa: For Comprehensions

Una **For Comprehension** (*comprensión for*) en Scala **no es un bucle imperativo**; es una forma de azúcar sintáctico para encadenar y componer secuencialmente llamadas a `map`, `flatMap` y `filter`. Esto hace que el código asíncrono o la lógica de transformación anidada sea mucho más legible, pareciéndose a un bucle secuencial, aunque conservando la inmutabilidad.

- **Palabra clave:** El uso de `yield` es obligatorio para que la comprensión retorne una nueva colección.

Ejemplo de Código (Búsqueda de Datos Anidada)

Imaginemos que tenemos una lista de usuarios y queremos obtener una tupla (`Nombre`, `Permiso`) para solo aquellos usuarios que tienen más de 18 años.

```
1  case class Usuario(nombre: String, edad: Int, permisos: List[String])
2
3  val usuarios = List(
4    Usuario("Alice", 25, List("read", "write")),
5    Usuario("Bob", 16, List("read"))
6  )
7
8  val permisosAdultos =
9    for {
10      // Generador (flatMap implícito) sobre la lista de usuarios
11      usuario <- usuarios
12
13      // Filtro (guarda) para incluir solo adultos
14      if usuario.edad >= 18
15
16      // Segundo Generador (map implícito) para iterar sobre los permisos
17      // del usuario
18      permiso <- usuario.permisos
19    } yield (usuario.nombre, permiso) // Resultado de cada iteración
20
21 // Resultado: List(("Alice", "read"), ("Alice", "write"))
22 // El elemento "Bob" fue eliminado por la guarda (filter).
```

5. Comparativa y Best Practices (*The Scala Way*)

Comparativa con Java y Python

Característica	Java (Imperativo/Streams)	Python (Comprensiones/Loops)	Scala (Funcional/HOF)
Filosofía	Mutabilidad por defecto; necesita <code>.stream().collect(...)</code> para PF/inmutabilidad.	Tipado dinámico; inmutabilidad opcional.	Inmutabilidad por defecto ; todas las operaciones devuelven una nueva colección
Transformación	Uso de <code>for</code> o <code>Stream.map</code> (a menudo verboso).	<code>map(lambda...)</code> o comprensiones de lista.	Uso de <code>.map</code> o <code>for</code> comprehension Sintaxis concisa (shorthand <code>_</code>).
Acoplamiento	Las operaciones de iteración (loops) acoplan la lógica del negocio con el control de flujo.	Las funciones de orden superior son valores de primera clase que pueden pasarse y componerse modularmente.	
Arrays vs Listas	Arrays (primitivos) vs. <code>ArrayList</code> / <code>LinkedList</code> (objetos).	<code>Array</code> (mutable en Scala) debe evitarse en favor de <code>List</code> o <code>Vector</code> .	

Best Practices

- Priorizar Inmutabilidad:** Utiliza siempre `List`, `Vector`, `Set` y `Map` inmutables. Si se necesita una colección que garantice una complejidad de acceso $O(1)$ y sea rápida, utiliza `Vector`.
- HOFs sobre Imperativo:** Evita los bucles `for` o `while` al estilo imperativo, ya que suelen implicar el uso de variables mutables (`var`). Utiliza `map`, `filter`, `flatMap` y `fold` para la manipulación de datos.
- Concisión con `_` y Lambdas:** Simplifica el código de las funciones anónimas (lambdas) utilizando el *placeholder* (`_`) siempre que el contexto de la expresión sea obvio.
- Uso de For Comprehensions:** Para encadenar múltiples operaciones de `flatMap` o incluir cláusulas de filtrado (`if`), utiliza las **For Comprehensions** para mejorar la legibilidad y claridad de la composición. Esto es un indicio de código avanzado ya que demuestra composición funcional.

El uso de colecciones funcionales es como usar una tubería modular para el flujo de agua: cada operación (`map` , `filter`) es una estación de procesamiento separada que recibe el flujo de datos y emite un nuevo flujo transformado, sin contaminar la tubería anterior ni la siguiente, garantizando la predictibilidad y seguridad del sistema.